Proves paquet mfpca

Leonel Fernando Nabaza Ruibal

Instal · lació

```
install.packages("MFPCA")
install.packages("funData")
install.packages("MASS")
```

```
library(funData)
```

```
##
## Adjuntando el paquete: 'funData'
```

```
## The following object is masked from 'package:stats':
##
##     integrate
```

```
library(MFPCA)
library(MASS)
```

Model de simulació

Generem n individus, cadascun amb un parell de funcions $X^{(1)}(t), X^{(2)}(t)$ definides sobre el mateix domini $t \in [0, 2\pi]$:

$$X^{(1)}(t) = \sin(t) + 0.5\,\epsilon_1(t), \qquad X^{(2)}(t) = 3\sin(t) + 0.5\,\epsilon_2(t), \qquad t \in [0, 2\pi]$$

amb l'error correlacionat a cada instant t:

$$\begin{pmatrix}\epsilon_1(t)\\ \epsilon_2(t)\end{pmatrix} \sim N\left(\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1 & \rho\\ \rho & 1\end{pmatrix}\right), \qquad \epsilon_i(t) \perp \epsilon_i(s) \text{ per a } t \neq s$$

És el mateix generador que fan servir Berrendero et al. (2011, eq. 5.7-5.8).

```
# Fixem la llavor per a reproduir sempre les mateixes dades simulades
set.seed(123)

n     <- 100                              # nombre d'individus
Ttime <- seq(0, 2*pi, length.out = 100)   # graella temporal
rho   <- 0.7                              # correlació entre els errors

# Matriu de covariàncies de l'error a cada instant t
Sigma <- matrix(c(1, rho, rho, 1), nrow = 2)

X1 <- matrix(NA, nrow = n, ncol = length(Ttime))
X2 <- matrix(NA, nrow = n, ncol = length(Ttime))
```

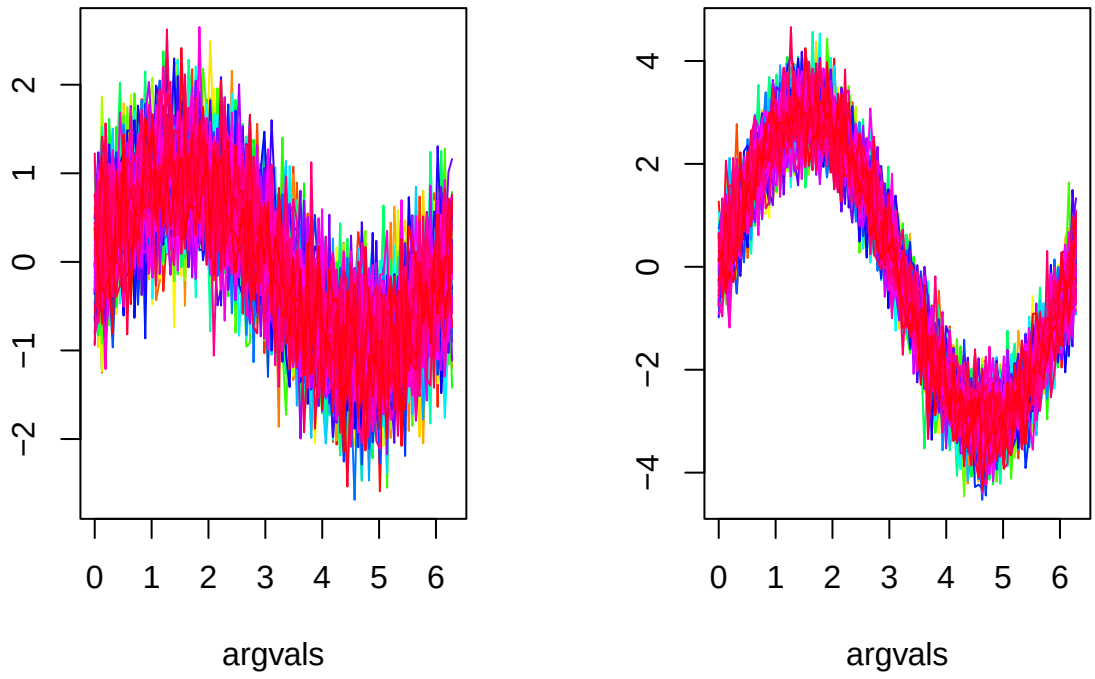
```r
# Generem cada corba individu a individu, punt a punt en t
for (i in 1:n) {
  for (j in seq_along(Ttime)) {
    eps <- mvrnorm(1, mu = c(0, 0), Sigma = Sigma)
    X1[i, j] <- sin(Ttime[j])   + 0.5 * eps[1]
    X2[i, j] <- 3*sin(Ttime[j]) + 0.5 * eps[2]
  }
}
```

Construcció dels objectes funcionals

```r
# Cada variable funcional es representa com un objecte funData
fd1 <- funData(argvals = Ttime, X = X1)
fd2 <- funData(argvals = Ttime, X = X2)

# Les combinem en un objecte multivariat (multiFunData)
mData <- multiFunData(fd1, fd2)

plot(mData)
```



Ajust de l'MFPCA

Cada variable s'expandeix en la seva pròpia base univariada (splines) i després es combinen:

$$X^{(j)}(t) = \mu^{(j)}(t) + \sum_{m=1}^{M_j} \rho_m^{(j)}\, \phi_m^{(j)}(t), \qquad j = 1, 2$$

seguit d'una PCA clàssica sobre els vectors de scores concatenats:

$$\rho_i = \left(\rho_{i,1}^{(1)}, \ldots, \rho_{i,M_1}^{(1)}, \rho_{i,1}^{(2)}, \ldots, \rho_{i,M_2}^{(2)}\right)$$

```
# Expansió univariada per a cadascuna de les dues variables funcionals
uniExpansions <- list(
  list(type = "splines1D", k = 10),
  list(type = "splines1D", k = 10)
)

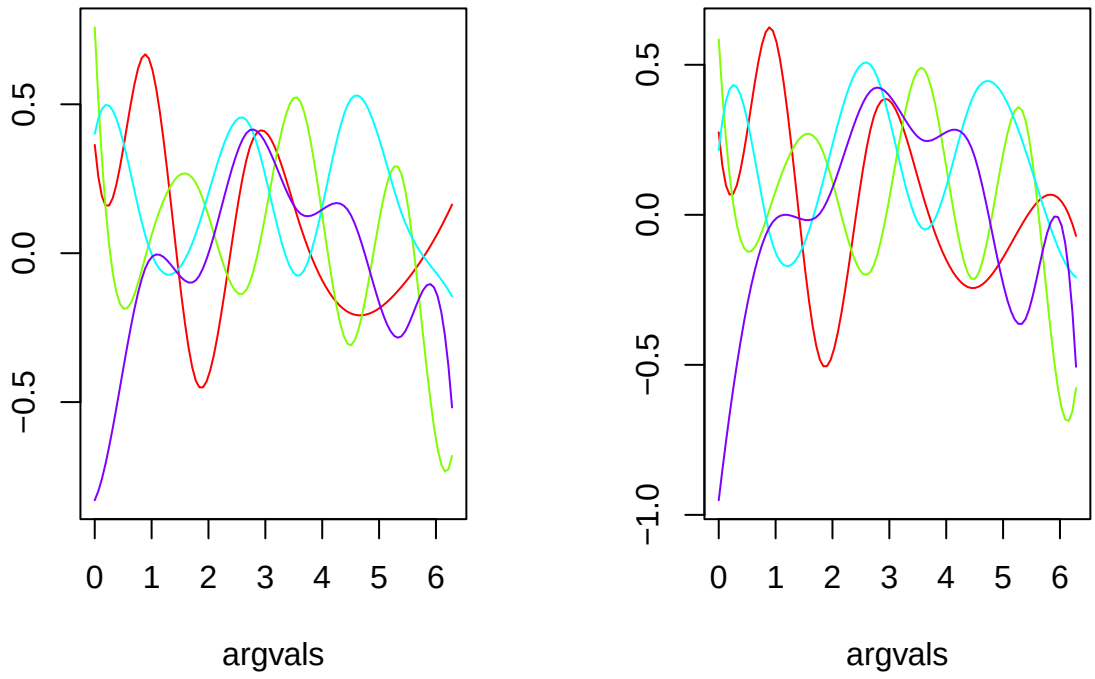
# M = nombre de components principals multivariades a estimar
mfpca_res <- MFPCA(mData, M = 4, uniExpansions = uniExpansions)
```

Resultats

```
# Variància explicada per cada component
mfpca_res$values
```

```
## [1] 0.04524643 0.03956792 0.03624884 0.02707229
```

```
# Funcions pròpies multivariades (components principals)
plot(mfpca_res$functions)
```



```r
# Scores (un valor per individu i per component)
head(mfpca_res$scores)
```

```
##              [,1]        [,2]       [,3]        [,4]
## [1,]  0.070242633 -0.06657212  0.1598652 -0.02932434
## [2,]  0.007097127  0.11458143  0.2729914  0.08574229
## [3,]  0.013700957 -0.14302054  0.4648058  0.19698464
## [4,] -0.436476728  0.02249932  0.2453126 -0.10229954
## [5,] -0.109027689  0.15700055 -0.2049394  0.15143409
## [6,] -0.061541681 -0.03363078  0.1027752  0.25147355
```

Model de simulació

A diferència del cas anterior, aquí les dues variables funcionals **no comparteixen el mateix domini temporal**. La primera, $X^{(1)}$, es defineix sobre $t \in [0, 2\pi]$, i la segona, $X^{(2)}$, sobre un domini completament diferent, $s \in [0, 10]$:

$$X^{(1)}(t) = \sin(t) + 0.3\,\epsilon_1(t), \qquad t \in [0, 2\pi]$$

$$X^{(2)}(s) = 0.5\,s + \cos(s) + 0.3\,\epsilon_2(s), \qquad s \in [0, 10]$$

on $\epsilon_1(t)$ i $\epsilon_2(s)$ són soroll blanc gaussià estàndard, **independent entre si**:

$$\epsilon_1(t) \sim N(0, 1), \qquad \epsilon_2(s) \sim N(0, 1)$$

Diferència important respecte al cas anterior: com que els dominis $[0, 2\pi]$ i $[0, 10]$ no coincideixen, no existeix cap instant t comú entre les dues variables. Per tant, ja no té sentit definir una correlació punt a punt ρ entre ϵ_1 i ϵ_2 (no hi ha un mateix t on avaluar-les alhora): cada error es genera de manera independent dins del seu propi domini.

Aquest és exactament l'escenari que el mètode de Berrendero et al. (2011) **no pot tractar** (assumeix un domini comú $[c, d]$ per a totes les variables), i que motiva la generalització de Happ & Greven (2018).

```r
set.seed(123)

n <- 100

# Variable 1: domini [0, 2*pi]
t1 <- seq(0, 2*pi, length.out = 80)
X1 <- matrix(NA, nrow = n, ncol = length(t1))
for (i in 1:n) {
  X1[i, ] <- sin(t1) + 0.3 * rnorm(length(t1))
}

# Variable 2: domini [0, 10] (rang i longitud de graella diferents)
t2 <- seq(0, 10, length.out = 50)
X2 <- matrix(NA, nrow = n, ncol = length(t2))
for (i in 1:n) {
  X2[i, ] <- 0.5 * t2 + cos(t2) + 0.3 * rnorm(length(t2))
}
```

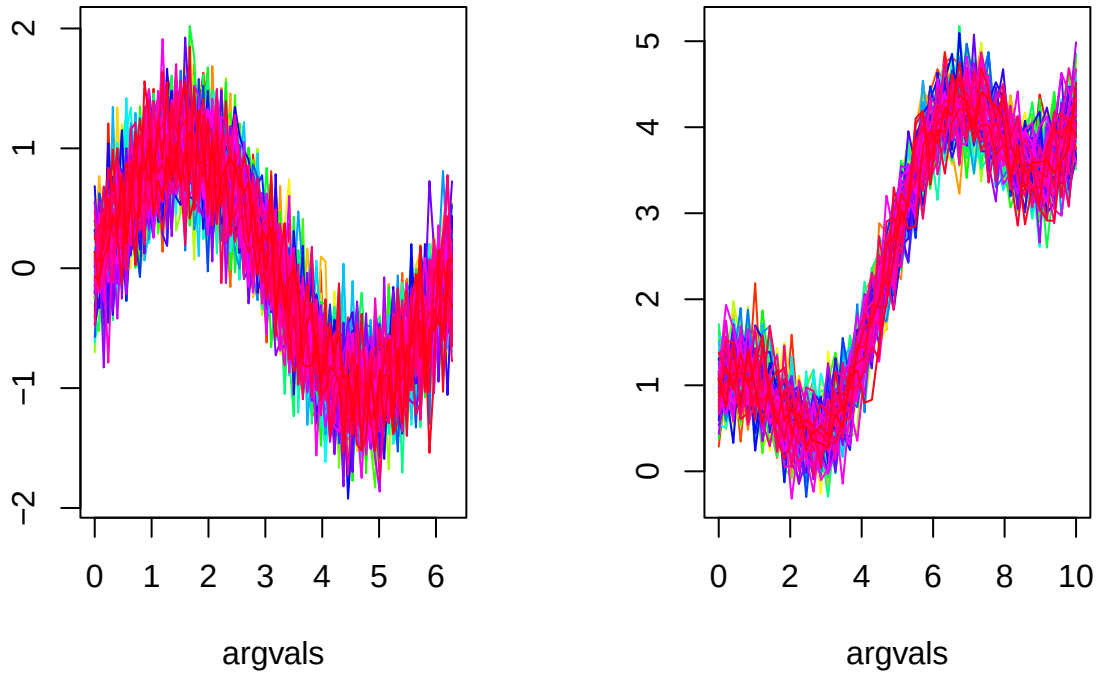
Construcció dels objectes funcionals

Cada variable es construeix amb la seva pròpia graella `argvals` (diferent longitud i diferent rang), i totes dues es combinen igualment en un `multiFunData`:

```
fd1 <- funData(argvals = t1, X = X1)
fd2 <- funData(argvals = t2, X = X2)

mData <- multiFunData(fd1, fd2)

plot(mData)
```



Ajust de l'MFPCA

El procediment intern és idèntic al cas de domini comú: cada variable s'expandeix per separat en la seva pròpia base (Karhunen-Loève univariat),

$$X^{(j)}(\cdot) = \mu^{(j)}(\cdot) + \sum_{m=1}^{M_j} \rho_m^{(j)}\, \phi_m^{(j)}(\cdot), \qquad j = 1, 2$$

i després es concatenen els scores i s'aplica una PCA clàssica:

$$\rho_i = \left(\rho_{i,1}^{(1)}, \ldots, \rho_{i,M_1}^{(1)}, \rho_{i,1}^{(2)}, \ldots, \rho_{i,M_2}^{(2)}\right)$$

El fet que $\phi_m^{(1)}$ estigui definida sobre $[0, 2\pi]$ i $\phi_m^{(2)}$ sobre $[0, 10]$ no suposa cap problema, perquè aquest pas final (la PCA clàssica) es fa sobre els **coeficients** $\rho_{i,m}^{(j)}$ (nombres reals), no sobre les funcions en si mateixes.

```r
uniExpansions <- list(
  list(type = "splines1D", k = 8),
  list(type = "splines1D", k = 8)
)

mfpca_res <- MFPCA(mData, M = 3, uniExpansions = uniExpansions)
```

Resultats

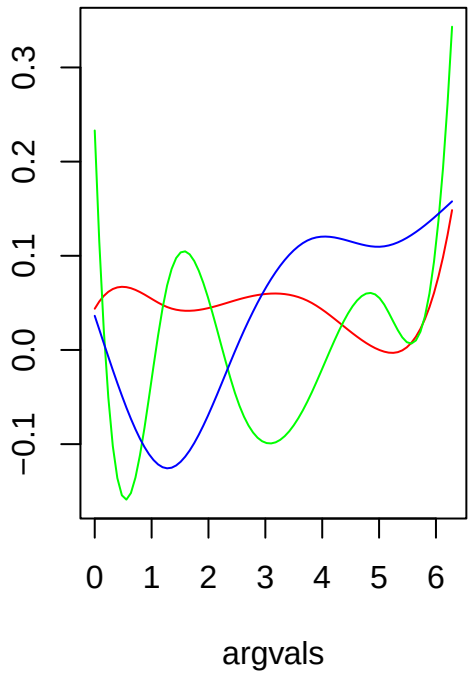
```r
# Variància explicada per cada component
mfpca_res$values
```

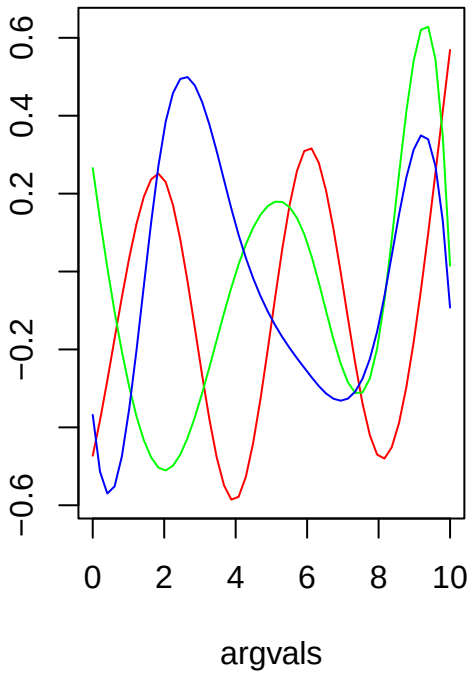
```
## [1] 0.02860096 0.02439292 0.02204028
```

```r
# Percentatge de variància explicada
mfpca_res$values / sum(mfpca_res$values) * 100
```

```
## [1] 38.11726 32.50908 29.37366
```

```r
# Funcions pròpies multivariades (una funció per a cada domini original)
plot(mfpca_res$functions)
```






```r
# Scores (un valor per individu i per component)
head(mfpca_res$scores)
```

```
##             [,1]        [,2]        [,3]
## [1,] -0.21140110 -0.03390790  0.11647452
```

```
## [2,]  0.32421164  0.18214165 -0.06137233
## [3,]  0.08819330  0.12565303  0.17961639
## [4,]  0.16780881 -0.08334875  0.10206631
## [5,]  0.11368072  0.07641094 -0.06885242
## [6,]  0.02279858 -0.14637522 -0.11544287
```